

HEP Agent Simulation Framework

Model Documentation

May 13, 2026

Contents

1	Introduction	2
2	Environment Model	3
2.1	Grid Structure	3
2.2	Habitat Suitability (HEP)	3
3	Agent Dynamics	4
3.1	Population Structure	4
3.2	Movement Rules	4
3.3	Demographics	4
3.3.1	Birth Model (Verhulst Pressure)	4
3.3.2	Death Model	5
4	Interaction Model	6
4.1	Resource Competition	6
5	Recent Changes	7
6	Project State Summary - May 5, 2026	8
6.1	Objective	8
6.2	Completed Work	8
6.2.1	1. Dead Agent Archive & NetCDF Storage (April 21st)	8
6.2.2	2. New Module Pattern & World Data Access (April 23rd)	8
6.2.3	3. DBSCAN, Convex Hull & Performance Metrics (April 28th)	9
6.2.4	4. Unified Density Smoothing & Global Metrics (April 29th)	9
6.2.5	5. Birth-Death Refactoring & Logic Consolidation (May 4th)	9
6.2.6	6. Architectural Refinement & Critical Fixes (April - May)	10
6.3	TODO later	10
6.3.1	7. Cluster Population Control & Multi-Population Graphing (May 13th)	10

Chapter 1

Introduction

This document describes the mathematical and logical model underlying the Human Ecology Project (HEP) Agent Simulation.

Chapter 2

Environment Model

2.1 Grid Structure

Description of the spatial grid, resolution, and coordinate system.

2.2 Habitat Suitability (HEP)

Description of the HEP index and how it influences agent survival and movement.

Chapter 3

Agent Dynamics

3.1 Population Structure

Agents are defined by a set of intrinsic and extrinsic attributes:

- **Age**: Tracked in ticks (1 tick $\approx$ 1 week).
- **Gender**: Binary classification (Male/Female).
- **Resources**: Energy levels accumulated from grid cells.

3.2 Movement Rules

Logic governing agent migration and dispersal.

3.3 Demographics

The demographics model manages population growth and mortality through a combination of individual agent rules and macroscopic feedback loops.

3.3.1 Birth Model (Verhulst Pressure)

To ensure population stability and prevent exponential runaway, the simulation utilizes a **macroscopic fertility scaling factor** ($K_{fertility}$).

The target birth rate ϕ_{target} is calculated using the Verhulst logistic equation:

$$\phi_{target} = r \cdot \left(1 - \frac{N}{N_C}\right) \quad (3.1)$$

Where:

- r : Intrinsic growth rate.
- N : Current total population count.
- N_C : Carrying capacity of the environment.

The actual fertility of each female agent is then scaled by $K_{fertility}$ to match this target, bounded by K_{min} and K_{max} to prevent demographic collapse or excessive volatility.

3.3.2 Death Model

Mortality is driven by multiple factors:

- **Natural Mortality:** Age-dependent risk.
- **Starvation:** Triggered when an agent's resources fall below a critical threshold.
- **Environmental Risk:** Probability of death based on HEP suitability (e.g., drowning in water cells).

Chapter 4

Interaction Model

4.1 Resource Competition

How agents compete for limited resources in a grid cell.

Chapter 5

Recent Changes

Chapter 6

Project State Summary - May 5, 2026

6.1 Objective

The primary goals addressed recently were the implementation of **Auto K-Means and DB-SCAN**, resolving simulation stability issues, performing a general codebase cleanup, and standardizing the **Birth-Death simulation modules** for consistent scientific modeling.

6.2 Completed Work

6.2.1 1. Dead Agent Archive & NetCDF Storage (April 21st)

- **Objective:** Implement a performance-conscious system for archiving deceased agents for historical analysis.
- **Work Done:**
 - **Fortran Backend:** Added `store_dead_agents` flag to `world_container` and logic to archive agent data before deletion.
 - **Asynchronous Pipeline:** Built an async Python writer using `multiprocessing.Queue` to persist data to NetCDF without blocking the simulation thread.
 - **UI Integration:** Added a checkbox in the "Full Simulation" tab and status indicators for the writer queue.
 - **Bug Fixes:** Resolved segfaults in the F2PY interface by refactoring agent extraction into a clean dataclass-based pipeline.

6.2.2 2. New Module Pattern & World Data Access (April 23rd)

- **Objective:** Standardize how new agent-logic modules are added and how they interact with global state.
- **Work Done:**
 - **Accumulator Reset:** Established a mechanism for `t_tick_accumulators` that automatically reset at the end of each simulation tick.
 - **Module Registration:** Defined a clear 3-step pattern for adding modules (Fortran ID -> Python Interface -> SpawnPointEditor registration) following the `reviewed_agent_motion` template.
 - **World Access:** Identified and documented accessible agent variables via the `world` object to facilitate custom agent-logic development.

6.2.3 3. DBSCAN, Convex Hull & Performance Metrics (April 28th)

- **Objective:** Expand clustering options, improve cluster integrity, and track simulation efficiency.
- **Work Done:**
 - **DBSCAN Implementation:** Added point-based and grid-based DBSCAN clustering in Fortran.
 - **UI Controls:** Added dynamic control over DBSCAN parameters (`eps`, `minPts`) directly from the Python GUI.
 - **Convex Hull Fill:** Integrated a convex hull post-processing step into the grid-based voting mechanism to eliminate holes and noise gaps within identified clusters.
 - **Performance Monitoring:** Implemented `avg_ms_per_tick` tracking. The average time per tick is now displayed in the window title and is available as a plottable variable in the time-series UI.

6.2.4 4. Unified Density Smoothing & Global Metrics (April 29th)

- **Objective:** Standardize density smoothing across all clustering modes and enable real-time plotting of global states.
- **Work Done:**
 - **Unified Smoothing:** Replaced redundant local smoothing in Auto K-Means with a global `human_density_smoothing_radius` parameter.
 - **Iterative Smoothing:** Added `human_density_smoothing_iterations` to allow for aggressive, multi-pass global smoothing.
 - **Global Metrics:** Exported `t_dynamic_state` and `t_tick_accumulators` to Python. Added variables like `k_fertility (sim)`, `phi_death_acc (sim)`, and various `death_*` (`sim`) debug counters to the UI plotting engine.
 - **Dual-Axis Fix:** Resolved a PyQtGraph closure bug where secondary axes caused the plot canvas to collapse.

6.2.5 5. Birth-Death Refactoring & Logic Consolidation (May 4th)

- **Objective:** Standardize parameter naming and consolidate biological logic for easier scientific review.
- **Work Done:**
 - **Parameter Schema Migration:** Replaced legacy `b1-b4` identifiers with standardized ecological parameters: `r` (growth rate), `NC` (carrying capacity), `Kmin`, and `Kmax`.
 - **Module Consolidation:** Migrated birth/death logic from the obsolete `mod_birth_death_new.f95` into the reviewed and consolidated `mod_reviewed_modules.f95`.
 - **Build System Sync:** Updated the dependency graph and Python interface to support the new parameter schema and modular structure.

6.2.6 6. Architectural Refinement & Critical Fixes (April - May)

- **K-Means Farthest-First Traversal:** Replaced the peak-value search in K-Means with a **Farthest First Traversal (K-means++)** subroutine to ensure geographically remote populations are isolated uniformly.
- **Cluster ID 1-Indexing:** Fixed a visualization bug where Cluster ID 0 was being masked as noise; the system now uses 1-based indexing for all valid clusters.
- **Rendering & Matrix Edge Geometry:** Resolved line rendering issues by translating cluster maps directly through NumPy boolean slicing filters and "burning" 1-pixel white boundaries into the RGBA output matrices.
- **3D Visualization:** Replaced unstable OpenGL 3D views with Matplotlib 3D surfaces in the `compare_smoothing.py` tool for robust cluster verification in headless environments.
- **F2PY Namespace Hotfix:** Fixed a critical bug where localized imports of `mod_python_interface` shadowed the extension module, causing `AttributeError` crashes in the update loop.

6.3 TODO later

- **[LATEX/DOCS]:** Add an entry explaining the Farthest-First Traversal algorithm.
- **[PYTHON/DOCS]:** Document that visual rendering loops for 2D and 3D are fundamentally separate.
- **[PYTHON/DOCS]:** Mark that the backend expects exactly 4-channel RGBA arrays for map updates.

6.3.1 7. Cluster Population Control & Multi-Population Graphing (May 13th)

- **Objective:** Resolve population convergence discrepancies between global and cluster-based modules and improve the Python graphing capabilities.
- **Work Done:**
 - **Divergence Analysis:** Identified that the cluster-based module (`new_birth`) was severely under-suppressing births compared to the global (`reviewed_birth`) module. This was caused by the cluster module relying on the per-cluster accumulator `n_alive_acc(jp)`, which structurally misses agents not assigned to any cluster (e.g. `c_idx = 0`), resulting in massive population under-estimation.
 - **Debug Tooling:** Implemented a global-vs-cluster diagnostic summation script within `python_interface.f95` to track the exact magnitude of unassigned agents escaping the control logic.
 - **Dynamic UI Plotting via Population Filters:**
 - Enhanced `mod_python_interface.get_dynamic_state_stats` and `get_cluster_info` to optionally accept and return stats based on a specific `jp` population index.
 - Added a dedicated `pop`: spinbox to the Python GUI (in `application.py`) that feeds into the graph configuration.
 - Implemented **"Dominant Population"** (`pop = -1`) logic, allowing graphs to dynamically query the backend tick-by-tick and automatically select and plot statistics (`NC`, `n_agents`, `k_fertility`) specifically for whichever population currently has the most agents within the targeted cluster or globally.